

Roteiro Prática 1

Samuel Tomazela

4 de setembro de 2026

1 Introdução

Relatório referente à Prática 1 da disciplina de Laboratório de Arquitetura e Organização de Computadores. Os códigos desenvolvidos estão detalhados a seguir, acompanhados de comentários explicativos. Todos os algoritmos foram testados com sucesso.

2 Códigos em Assembly MIPS

2.1 Exercício 1: Menu

```
1 .data
2 msgMenu: .asciiz "\n----- Menu -----\n(1)    igual?\n(2) Somat rio\n(3)
   Sequencia\n(4) Sair\nDigite qual op    o deseja: "
3
4 sequenceMsg: .asciiz "Digite a quantidade de numeros da sequencia que deseja
   ver: "
5
6 somatoryMsg: .asciiz "Digite o numero para o somatorio: "
7
8 isEqualFirstMsg: .asciiz "Digite o primeiro numero: "
9 isEqualSecondMsg: .asciiz "Digite o segundo numero: "
10
11 .text
12 start:
13 addi $v0, $zero, 4
14 la $a0, msgMenu #Imprime mensagem do Menu
15 syscall
16
17 addi $v0, $zero, 5
18 syscall
19
20 beq $v0, $zero, start # v0 == 0?
21 addi $t1, $zero, 1 #Inicializando variavel com 1
22
23 slt $t0, $v0, $zero # v0 < 0?
24 beq $t1, $t0, start # Sim, Ent o volta
25
26 addi $t2, $zero, 4 # v0 > 4?
27 slt $t0, $t2, $v0 # Sim, Ent o volta
28 beq $t1, $t0, start
29
30 addi $t0, $zero, 1
31 beq $v0, $t0, case1 # Case 1
32
33 addi $t0, $t0, 1
34 beq $v0, $t0, case2 # Case 2
35
36 addi $t0, $t0, 1
```

```

37 beq $v0, $t0, case3 # Case 3
38
39 addi $t0, $t0, 1
40 beq $v0, $t0, case4 # Case 4
41
42 case1:
43 addi $v0, $zero, 4 # Le string
44 la $a0, isEqualFirstMsg # a0 contem endere o de onde est a mensagem
45 syscall
46
47 addi $v0, $zero, 5 # Le inteiro
48 syscall
49
50 add $t0, $v0, $zero #Guardando inteiro lido
51
52 addi $v0, $zero, 4 # Le uma string
53 la $a0, isEqualSecondMsg
54 syscall
55
56 addi $v0, $zero, 5 #Le inteiro
57 syscall
58
59 add $t1, $v0, $zero
60 jal isEqualCode
61 addi $v0, $zero, 1 # Imprime inteiro
62 syscall
63
64 j start
65
66 case2:
67 addi $v0, $zero, 4 # Le string
68 la $a0, somatoryMsg # a0 contem endere o de onde est a mensagem
69 syscall
70
71 addi $v0, $zero, 5 # Le inteiro
72 syscall
73
74 add $t0, $v0, $zero # Guardando inteiro lido
75 jal somatoryCode
76 addi $v0, $zero, 1 # Imprime inteiro
77 syscall
78 j start
79
80 case3:
81 addi $v0, $zero, 4 # L string
82 la $a0, sequenceMsg # a0 contem endere o de onde est a mensagem
83 syscall
84
85 addi $v0, $zero, 5 # L inteiro
86 syscall
87
88 add $t0, $v0, $zero # Guardando inteiro lido
89
90 jal sequenceCode
91 addi $v0, $zero, 1 # Imprime inteiro
92 syscall
93
94 j start
95
96 case4: #Sair
97 j finaliza
98

```

```

99
100 isEqualCode:
101 sub $t2, $t0, $t1
102 beq $t2, $zero, isEqualTrue
103     add $a0, $zero, $zero
104     j isEqualFinal
105
106 isEqualTrue:
107     addi $a0, $zero, 1
108
109 isEqualFinal:
110 jr $ra
111
112
113 somatoryCode:
114 add $t1, $zero, $zero # Zerando variavel para o somatorio
115 somatoryLoop:
116     sub $t0, $t0, 1 #Subtraindo 1 do valor lido (Primeiro valor menor que
117         ele)
118     beq $t0, $zero, somatoryFinal
119     add $t1, $t0, $t1
120     j somatoryLoop
121
122 somatoryFinal:
123 add $a0, $t1, $zero
124 jr $ra
125
126 sequenceCode:
127 add $t1, $zero, $zero # Zerando variavel para guardar o numero da sequencia
128
129 beq $t2, $t0, sequenceFinal
130 addi $t1, $zero, 1 #Variavel que guarda o numero da sequencia
131 addi $t2, $zero, 1 #Numero que guarda em qual numero da sequencia esta
132 beq $t2, $t0, sequenceFinal
133 addi $t1, $t1, 1
134 addi $t2, $t2, 1
135 sequenceLoop:
136     beq $t2, $t0, sequenceFinal
137     addi $t1, $t1, 2
138     addi $t2, $t2, 1
139     j sequenceLoop
140 sequenceFinal:
141 add $a0, $t1, $zero
142 jr $ra
143
144 finaliza:
145 addi $v0, $zero, 10 # Finaliza
146 syscall

```

2.2 Exercício 2: BubbleSort

```

1 .data
2 firstMsg: .asciiz "Informe um numero: "
3 secondMsg: .asciiz "Novos Valores de V: "
4 thirdMsg: .asciiz " "
5 vetor: .word 0:9
6
7 .text
8 la $t0, vetor

```

```

9  addi $s0, $zero, 10 #Guardando tamanho do vetor
10 add $t1, $zero, $zero #Variavel que vai andar pelo vetor
11 addi $t2, $zero, 4 #Variavel para andar pela memoria
12
13 loopLeitura:
14     beq $t1, $s0, fimLeitura
15     addi $v0, $zero, 4 # L      string
16     la $a0, firstMsg # a0 contem endere  o de onde esta a mensagem
17     syscall
18
19     addi $v0, $zero, 5 # L      inteiro
20     syscall
21
22     sw $v0, 0($t0)
23     add $t0, $t0, $t2
24     addi $t1, $t1, 1
25     j loopLeitura
26
27 fimLeitura:
28 add $t1, $zero, 1 #Variavel que vai andar pelo vetor
29 add $t3, $zero, $zero #Variavel 2 que vai andar pelo vetor
30 addi $s1, $zero, 1 #Guarda o valor 1 para compara  o no beq
31
32 loopBubble:
33 la $t0, vetor
34 beq $t1, $s0, fim
35 add $t3, $zero, $zero
36 sub $t4, $s0, $t1      # limite de j
37 loopBubble2:
38 beq $t3, $t4, fimBubble
39 lw $t5, 0($t0) # vetor[j]
40 add $t0, $t0, $t2
41 lw $t6, 0($t0) # vetor[j+1]
42
43 sle $t7, $t6, $t5
44 beq $t7, $s1, fimBubble2
45
46 sw $t5, 0($t0)
47 sub $t0, $t0, $t2
48 sw $t6, 0($t0)
49 add $t0, $t0, $t2
50
51 fimBubble2:
52 addi $t3, $t3, 1 # j++
53 j loopBubble2
54
55 fimBubble:
56 addi $t1, $t1, 1 # i++
57 j loopBubble
58
59
60
61 fim:
62 addi $v0, $zero, 4 # L      string
63 la $a0, secondMsg # a0 contem endere  o de onde est      a mensagem
64 syscall
65
66
67 la $t0, vetor
68 add $t1, $zero, $zero #Variavel que vai andar pelo vetor
69
70 loop3:

```

```

71     beq $t1, $s0, fim3
72
73     lw $a0, 0($t0)
74     addi $v0, $zero, 1 # Imprime inteiro
75     syscall
76
77     addi $v0, $zero, 4 # L      string
78     la $a0, thirdMsg # a0 contem endere  o de onde est      a mensagem
79     syscall
80
81     add $t0, $t0, $t2
82     addi $t1, $t1, 1
83     j loop3
84
85 fim3:
86 addi $v0, $zero, 10 # Finaliza
87 syscall

```

2.3 Exercício 3: Calculadora

```

1  .data
2  msgMenu: .asciiz "\n----- Menu ----- \n (1)Soma\n (2)Subtra  o\n (3)
      Multiplica  o\n (4)Exponencial \n (5)Sair\nDigite qual op  o deseja: "
3
4  firstNumber: .asciiz "Digite o primeiro n mero: "
5  secondNumber: .asciiz "Digite o segundo n mero: "
6  .text
7
8  start:
9  addi $v0, $zero, 4
10 la $a0, msgMenu #Imprime mensagem do Menu
11 syscall
12
13 addi $v0, $zero, 5
14 syscall
15
16 beq $v0, $zero, start # v0 == 0?
17 addi $t1, $zero, 1 #Inicializando variavel com 1
18
19 slt $t0, $v0, $zero # v0 < 0?
20 beq $t1, $t0, start # Sim, Ent o volta
21
22 addi $t2, $zero, 5 # v0 > 5?
23 slt $t0, $t2, $v0 # Sim, Ent o volta
24 beq $t1, $t0, start
25
26 addi $t0, $zero, 1
27 beq $v0, $t0, case1 # Case 1
28
29 addi $t0, $t0, 1
30 beq $v0, $t0, case2 # Case 2
31
32 addi $t0, $t0, 1
33 beq $v0, $t0, case3 # Case 3
34
35 addi $t0, $t0, 1
36 beq $v0, $t0, case4 # Case 4
37
38 addi $t0, $t0, 1
39 beq $v0, $t0, case5 # Case 5

```

```

40
41 case1:
42 jal getData
43 jal plusCode
44 jal printResult
45 j start
46
47 case2:
48 jal getData
49 jal subtractionCode
50 jal printResult
51 j start
52
53 case3:
54 jal getData
55 jal timesCode
56 jal printResult
57 j start
58
59 case4:
60 jal getData
61 jal exponentialCode
62 jal printResult
63 j start
64 case5: #Sair
65 j finaliza
66
67 plusCode:
68 add $a0, $t0, $t1 # n1 + n2 guardado em a0
69 jr $ra
70
71 subtractionCode:
72 sub $a0, $t0, $t1 # n1 - n2 guardado em a0
73 jr $ra
74
75 timesCode:
76 add $t2, $zero, $zero # Zerando variavel para guardar o numero da sequencia
77 addi $a0, $zero, 0 # Zerando a variavel aoande fica o resultado
78
79 timesLoop:
80 beq $t1, $t2, timesFinal # Enquanto i diferente de n2
81 addi $t2, $t2, 1 #Variavel i
82 add $a0, $t0, $a0 # resultado += n1
83 j timesLoop
84 timesFinal:
85 jr $ra
86
87 exponentialCode:
88 add $t6, $zero, $ra
89 addi $t5, $zero, 1
90 add $t3, $t1, $zero
91 addi $t4, $zero, 1 # i do exponencial
92 add $t1, $t0, $zero #n2 = n1 para poder reutilizar o codigo de multiplica o
93
94 beq $t3, $t5, exceptionalFinal
95 add $t5, $zero, $zero
96 exponentialLoop:
97 beq $t3, $t4, exponentialFinal
98 jal timesCode
99 addi $t4, $t4, 1
100 add $t5, $a0, $zero
101 add $t0, $t5, $zero

```

```

102 j exponentialLoop
103 exponentialFinal:
104 add $a0, $t5, $zero
105 add $ra, $t6, $zero
106 jr $ra
107
108 exceptionalFinal:
109 add $a0, $t0, $zero
110 add $ra, $t6, $zero
111 jr $ra
112
113 getData:
114 addi $v0, $zero, 4 # Le string
115 la $a0, firstNumber # a0 contém endereço de onde está a mensagem
116 syscall
117
118 addi $v0, $zero, 5 # Le inteiro
119 syscall
120
121 add $t0, $v0, $zero #Guardando inteiro lido
122
123 addi $v0, $zero, 4 # Le uma string
124 la $a0, secondNumber
125 syscall
126
127 addi $v0, $zero, 5 #Le inteiro
128 syscall
129
130 add $t1, $v0, $zero #Guardando inteiro lido
131 jr $ra
132
133 printResult:
134 addi $v0, $zero, 1 # Imprime inteiro
135 syscall
136 jr $ra
137
138 finaliza:
139 addi $v0, $zero, 10 # Finaliza
140 syscall

```

3 Provação - Resultados dos Testes

3.1 Exercício 1: Menu

O primeiro exercício consiste na implementação de um menu de opções, permitindo ao usuário selecionar entre a verificação de igualdade, o cálculo do somatório e a geração de uma sequência numérica.

3.1.1 Opção 1: É Igual?

As Figuras apresentam os resultados da execução da opção de verificação de igualdade entre dois números, demonstrando tanto o caso em que os valores informados são iguais quanto o caso em que os valores são diferentes.

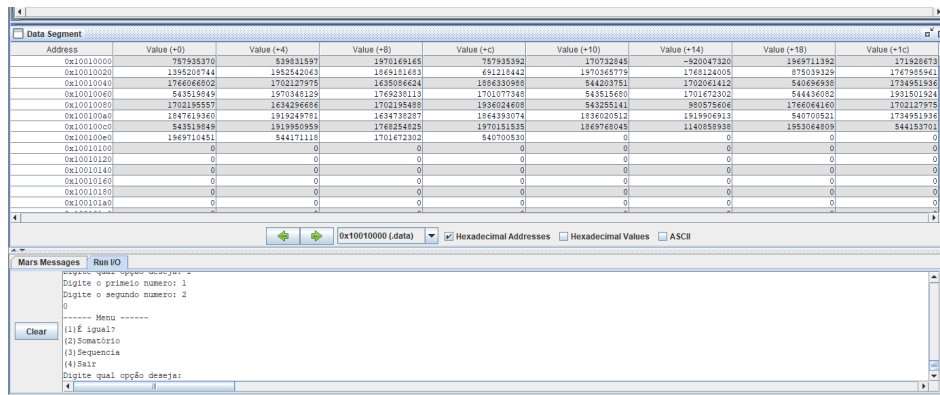


Figura 1: Resultado da execução do Exercício 1 (Menu), opção É Igual? com valores iguais.

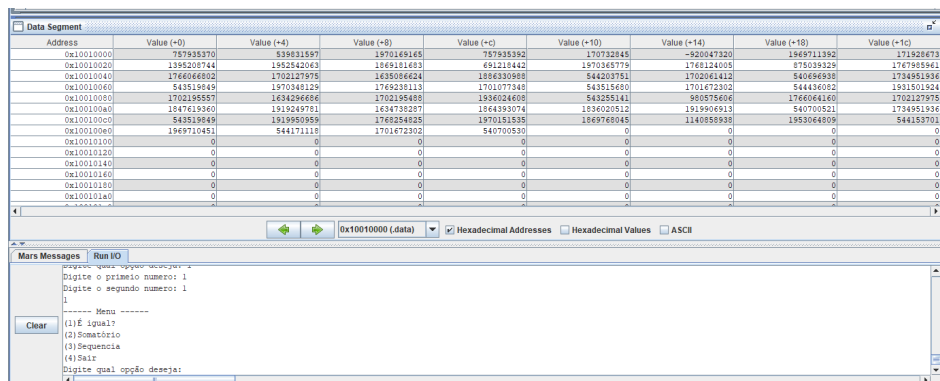


Figura 2: Resultado da execução do Exercício 1 (Menu), opção É Igual? com valores diferentes.

3.1.2 Opção 2: Somatório

A Figura apresenta o resultado da execução da opção de somatório, demonstrando o cálculo realizado a partir do número informado pelo usuário.

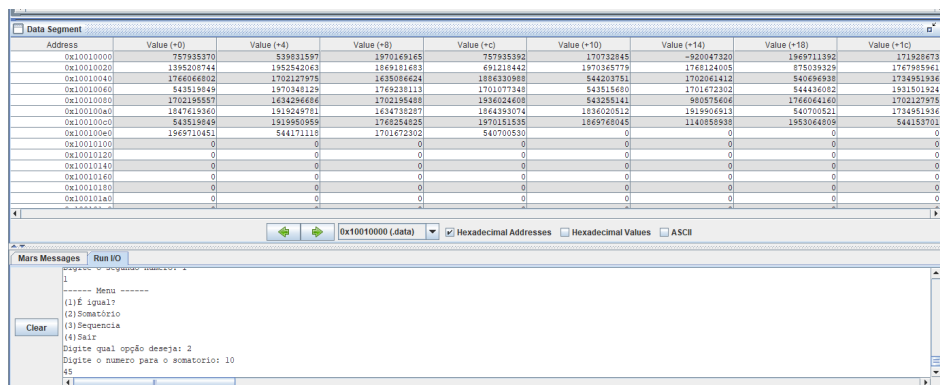


Figura 3: Resultado da execução do Exercício 1 (Menu), opção Somatório.

3.1.3 Opção 3: Sequência

A Figura apresenta o resultado da execução da opção de geração da sequência numérica, conforme a quantidade de termos solicitada pelo usuário.

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	787935370	539831597	1970169165	787935392	170722845	-520047320	1969711392	171928678
0x10010020	1395207744	1952542063	1869151693	691218442	1970360779	1768124005	875039329	1767985961
0x10010040	1766666802	1702127975	1635086624	1886339888	544203751	1702061412	540696938	1734951936
0x10010060	543519449	1970348129	1769238113	1701077340	543515460	1701072302	544436082	1931501924
0x10010080	1702195557	1634296686	1702195498	1896024608	543255141	805975604	1766066160	1702127975
0x100100A0	1047619360	1919249781	1634738287	1864393074	1836020512	1915906913	540700521	1734951936
0x100100C0	543519849	1919950959	1768254825	1970151535	1869768045	1140585938	1953064809	544153701
0x100100E0	1969710491	544171115	1701672302	540700530	0	0	0	0
0x10010100	0	0	0	0	0	0	0	0
0x10010120	0	0	0	0	0	0	0	0
0x10010140	0	0	0	0	0	0	0	0
0x10010160	0	0	0	0	0	0	0	0
0x10010180	0	0	0	0	0	0	0	0
0x100101A0	0	0	0	0	0	0	0	0

Figura 4: Resultado da execução do Exercício 1 (Menu), opção Sequência.

3.2 Exercício 2: BubbleSort

O segundo exercício consiste na implementação do algoritmo Bubble Sort para ordenar, em ordem decrescente, os 10 números informados pelo usuário.

A Figura apresenta o resultado da execução do algoritmo após a ordenação dos valores.

```

Informe um numero: 10
Informe um numero: 2
Informe um numero: 3
Informe um numero: 5
Informe um numero: 6
Informe um numero: 8
Informe um numero: 11
Informe um numero: 2
Informe um numero: 3
Informe um numero: 4
Novos Valores de Vi: 11 10 8 6 5 4 3 3 2 2

```

Figura 5: Resultado da execução do Exercício 2 (BubbleSort).

3.3 Exercício 3: Calculadora

O terceiro exercício consiste na implementação de uma calculadora com operações de soma, subtração, multiplicação e exponenciação.

3.3.1 Opção 1: Soma

A Figura apresenta o resultado da execução da operação de soma, demonstrando o valor obtido a partir dos dois números informados pelo usuário.

```

----- Menu -----
(1) Soma
(2) Subtração
(3) Multiplicação
(4) Exponencial
(5) Sair
Digite qual opção deseja: 1
Digite o primeiro número: 14
Digite o segundo número: 15
29

```

Figura 6: Resultado da execução do Exercício 3 (Calculadora), operação Soma.

3.3.2 Opção 2: Subtração

A Figura apresenta o resultado da execução da operação de subtração, demonstrando o valor obtido a partir dos dois números informados pelo usuário.

```
----- Menu -----  
(1) Soma  
(2) Subtração  
(3) Multiplicação  
(4) Exponencial  
(5) Sair  
Digite qual opção deseja: 2  
Digite o primeiro número: 19  
Digite o segundo número: 13  
6
```

Figura 7: Resultado da execução do Exercício 3 (Calculadora), operação Subtração.

3.3.3 Opção 3: Multiplicação

A Figura apresenta o resultado da execução da operação de multiplicação, realizada por meio de somas sucessivas dos números informados.

```
----- Menu -----  
(1) Soma  
(2) Subtração  
(3) Multiplicação  
(4) Exponencial  
(5) Sair  
Digite qual opção deseja: 3  
Digite o primeiro número: 7  
Digite o segundo número: 8  
56
```

Figura 8: Resultado da execução do Exercício 3 (Calculadora), operação Multiplicação.

3.3.4 Opção 4: Exponencial

A Figura apresenta o resultado da execução da operação de exponenciação, realizada por meio de multiplicações sucessivas dos valores informados.

```
56
----- Menu -----
(1) Soma
(2) Subtração
(3) Multiplicação
(4) Exponencial
(5) Sair
Digite qual opção deseja: 4
Digite o primeiro número: 3
Digite o segundo número: 4
81
----- Menu -----
(1) Soma
(2) Subtração
(3) Multiplicação
(4) Exponencial
(5) Sair
Digite qual opção deseja: 5

-- program is finished running --
```

Figura 9: Resultado da execução do Exercício 3 (Calculadora), operação Exponencial.